

EXPERIMENT 5

AIM: i. Create a simple PL/SQL program which includes declaration section, executable section and exception –Handling section (Ex. Student marks can be selected from the table and printed for those who secured first class and an exception can be raised if no records were found)

ii. Insert data into student table and use COMMIT, ROLLBACK and SAVEPOINT in PL/SQL block.

SOLUTION:

PL/SQL stands for Procedural Language extension of SQL.

It combines SQL commands with programming constructs.

Used to write program logic inside database.

PL/SQL programs are written in blocks.

Block structure has three parts:

- Declaration section (optional)
- Execution section (mandatory)
- Exception handling section (optional)

Variables are used to store data.

Output is displayed using DBMS_OUTPUT.PUT_LINE.

Sample PL/SQL Block looks:

DECLARE

declaration statements; Optional

BEGIN

executable statements Mandatory

EXCEPTION

exception handling statements Optional

END;

Comments:

- 1) Single line Comments: --(double hyphen) is used for single line comments
- 2) Multiple line Comments: /*...*/ symbols are used for multi line comments

PL/SQL Variables

A **variable** is a name used to **store a value**. The value **can change** during program execution.

Syntax to declare a variable:

variable_name datatype [NOT NULL]:= value];

Examples:

marks NUMBER;

name VARCHAR2(20) := 'Ravi';

Program:

```
SET SERVEROUTPUT ON;

DECLARE

    v_count NUMBER := 0;

BEGIN

    FOR s IN (SELECT rollno, name, marks

        FROM student

        WHERE marks >= 60)

    LOOP

        v_count := v_count + 1;

        DBMS_OUTPUT.PUT_LINE('RollNo: ' || s.rollno || ', ' ||

            ' Name: ' || s.name || ', ' ||

            ' Marks: ' || s.marks);

    END LOOP;

    IF v_count = 0 THEN

        RAISE NO_DATA_FOUND;

    END IF;

    EXCEPTION

    WHEN NO_DATA_FOUND THEN

        DBMS_OUTPUT.PUT_LINE('No Record Found');

    WHEN OTHERS THEN

        DBMS_OUTPUT.PUT_LINE('Some error occurred');

END;

/
```

Output:

```
RollNo: 1 , Name: ravi , Marks: 95

RollNo: 2 , Name: sita , Marks: 88

RollNo: 3 , Name: kiran , Marks: 82

RollNo: 4 , Name: anil , Marks: 75

RollNo: 5 , Name: ram , Marks: 65

RollNo: 9 , Name: john , Marks: 90
```

PL/SQL procedure successfully completed.

ii. Insert data into student table and use **COMMIT**, **ROLLBACK** and **SAVEPOINT** in PL/SQL block.

1. COMMIT

Saves the **permanent changes** made in the database.

After COMMIT, changes cannot be undone.

Example

UPDATE student SET marks = 80 WHERE rollno = 1;

COMMIT;

✓ Means → Save changes permanently.

2. ROLLBACK

Cancels the **recent changes** made.

Used when error occurs or you want to undo operations.

Example

DELETE FROM student WHERE rollno = 1;

ROLLBACK;

✓ Means → Undo changes.

3. SAVEPOINT

Creates a **checkpoint** inside a transaction.

You can rollback to a specific point.

Example

SAVEPOINT A;

UPDATE student SET marks = 70 WHERE rollno = 2;

ROLLBACK TO A;

✓ Undo all changes done **after SAVEPOINT A**.

✓ Means → UPDATE operation is cancelled.

✓ But changes before SAVEPOINT remain safe.

Program:

```
SET SERVEROUTPUT ON;
```

```
BEGIN
```

```
-- First insert
```

```
INSERT INTO student VALUES (101,'aaa',72,null);
```

```
DBMS_OUTPUT.PUT_LINE('Inserted aaa');
```

```
SAVEPOINT sp1;
```

```
-- Second insert
```

```
INSERT INTO student VALUES (102,'bbb',55,null);

DBMS_OUTPUT.PUT_LINE('Inserted bbb');

SAVEPOINT sp2;

-- Third insert

INSERT INTO student VALUES (103,'ccc',68,null);

DBMS_OUTPUT.PUT_LINE('Inserted CCC');

-- Rollback to SAVEPOINT sp2 (undo only third insert)

ROLLBACK TO sp2;

DBMS_OUTPUT.PUT_LINE('Rollback to SAVEPOINT sp2');

-- Commit remaining changes

COMMIT;

DBMS_OUTPUT.PUT_LINE('Transaction committed');

EXCEPTION

    WHEN OTHERS THEN

        ROLLBACK;

        DBMS_OUTPUT.PUT_LINE('Error occurred. All changes rolled back');

END;
```

/

Output:

Inserted aaa

Inserted bbb

Inserted ccc

Rollback to SAVEPOINT sp2

Transaction committed

PL/SQL procedure successfully completed.